

Instalación inicial de MCP para trabajar con GitHub y RabbitMQ dentro del proyecto

Antes de empezar a usar MCP dentro del proyecto, es importante entender qué papel tiene. MCP no es una librería más del repositorio ni una parte de Laravel, sino un protocolo que permite que herramientas de IA como Claude, Gemini o aplicaciones construidas con OpenAI se conecten a sistemas externos de forma estandarizada. En este caso, lo que nos interesa es conectarlas con GitHub para trabajar sobre el repositorio y, si queremos ir un paso más allá, con RabbitMQ para reaccionar a eventos del proyecto de forma desacoplada.

Ejemplo de contexto real:

El developer no le pide a la IA "explícame qué es una pull request", sino "créame una issue para el chat del juego, revisa la PR y dime si falta validación".

Primer paso: preparar el entorno del proyecto

Para trabajar bien, el proyecto debe seguir teniendo su estructura normal en GitHub, tal como ya se indica en la práctica: repositorio, ramas, issues, pull requests y automatizaciones básicas. La novedad aquí es que añadimos una capa de asistencia inteligente encima del flujo de trabajo, no para reemplazarlo, sino para reforzarlo. Esto encaja especialmente bien con la parte de organización del proyecto y del trabajo en equipo que ya aparece en la práctica.

Ejemplo de comando real para preparar el repositorio local:

```
git clone https://github.com/tu-organizacion/plataforma-juegos.git
cd plataforma-juegos
git checkout -b feature/chat-tiempo-real
```

Instalar el MCP oficial de GitHub

Para GitHub, la opción más sólida ahora mismo es el servidor oficial github/github-mcp-server, que permite a herramientas de IA leer repositorios y ficheros, gestionar issues y pull requests, analizar código y automatizar flujos sobre GitHub. GitHub documenta además una imagen pública de Docker para ejecutarlo, y también permite limitar el alcance mediante toolsets o herramientas concretas, algo importante para no dar más permisos de los necesarios.

Ejemplo de comando real con Docker:

```
docker run -i --rm \
  -e GITHUB_PERSONAL_ACCESS_TOKEN=tu_token_github \
  -e GITHUB_TOOLSETS="repos,issues,pull_requests" \
  ghcr.io/github/github-mcp-server
```

Contexto real de uso:

La IA puede leer el repositorio, abrir una issue para "integrar RabbitMQ", revisar una PR del microservicio Python y sugerir si falta documentación o validación.

Token y permisos mínimos para GitHub

Para que esto funcione, hace falta un GitHub Personal Access Token. GitHub indica expresamente que el servidor MCP puede usar muchas APIs de GitHub y que conviene dar solo los permisos con los que te sientas cómodo. En una práctica docente o en un proyecto de aula, lo razonable es empezar con permisos limitados para repositorio, issues y pull requests, y no abrir acceso a más hasta que realmente haga falta.

Ejemplo de variable de entorno en local:

```
export GITHUB_PERSONAL_ACCESS_TOKEN=ghp_XXXXXXXXXXXXXXXXXXXX
```

Ejemplo de contexto real:

Si el developer solo necesita crear issues y revisar PRs, no tiene sentido darle acceso a acciones destructivas o a más repositorios de los necesarios.

Conectar GitHub MCP con Claude Code

Claude Code se puede usar desde terminal, VS Code, Cursor y otras superficies, y su documentación indica tanto la integración en VS Code como la conexión con herramientas externas mediante MCP. Para un proyecto académico o profesional pequeño, Claude Code en VS Code suele ser la forma más clara de empezar, porque el developer ve el repositorio, la conversación y los cambios en el mismo entorno.

Ejemplo de instalación de Claude Code en terminal:

```
curl -fsSL https://claude.ai/install.sh | bash
```

Ejemplo de contexto real:

*El developer abre el proyecto Laravel en VS Code, lanza Claude Code y le pide:
"revisa esta rama y dime si la nueva API de sesiones de juego rompe algo del chat".*

Configuración de Claude para usar el MCP de GitHub

La configuración exacta puede variar según el cliente MCP, pero GitHub documenta una forma típica de declarar el servidor con docker y pasarle el token mediante variables de entorno. La idea importante para la práctica no es memorizar el JSON, sino entender que Claude no “sabe” GitHub por magia: se le conecta explícitamente un servidor MCP que le da acceso controlado al repositorio.

Ejemplo de configuración:

```
{
  "servers": {
    "github": {
      "command": "docker",
      "args": [
        "run",
        "-i",
        "--rm",
        "-e",
        "GITHUB_PERSONAL_ACCESS_TOKEN",
        "ghcr.io/github/github-mcp-server"
      ],
      "env": {
        "GITHUB_PERSONAL_ACCESS_TOKEN": "ghp_XXXXXXXXXXXXXXXXXXXX"
      }
    }
  }
}
```

Ejemplo de contexto real:

Claude puede recibir la instrucción:

"crea una issue para separar mejor web.php y api.php y proponme checklist de terminado".

Conectar el proyecto con Gemini

Gemini CLI también soporta servidores MCP locales o remotos, y Google documenta tanto el uso general de MCP con Gemini CLI como la conexión a servidores remotos. Esto permite que el mismo proyecto pueda trabajarse con otra herramienta sin cambiar la arquitectura: el servidor MCP sigue siendo el mismo, lo que cambia es el cliente que lo consume.

Ejemplo de contexto real:

Un equipo puede usar Claude Code para revisar PRs y otro Gemini CLI para explorar el repositorio o lanzar tareas más guiadas desde terminal.

Ejemplo de comando real relacionado con MCP en Gemini:

```
npx add-mcp "https://gemini-api-docs-mcp.dev"
```

Ese ejemplo es del MCP público de documentación de Gemini, pero sirve para entender el patrón: el cliente añade un servidor MCP y desde ese momento puede usarlo dentro de su flujo de trabajo.

Y con OpenAI, cómo encaja en el proyecto

En OpenAI el encaje actual no suele hacerse como “instalar una app de escritorio y pegar un JSON” del mismo modo que en Claude Code o Gemini CLI, sino integrando MCP dentro de una aplicación propia mediante la Responses API, que ya soporta remote MCP servers. Esto significa que en un proyecto real puedes tener una pequeña capa propia, por ejemplo un panel interno o un asistente del equipo, que use OpenAI y consuma el mismo servidor MCP remoto para trabajar con GitHub o con RabbitMQ.

Ejemplo de contexto real:

El centro o la empresa puede tener un asistente interno basado en OpenAI que diga: "lista las PR abiertas del proyecto de juegos, resume riesgos y crea una issue de refactor".

Ejemplo de pseudocontexto técnico:

App propia + OpenAI Responses API + remote MCP GitHub server

Instalar RabbitMQ para añadir una capa de eventos al proyecto

Una vez que GitHub ya está conectado mediante MCP, se puede añadir RabbitMQ como ampliación avanzada del proyecto. Aquí la idea no es que RabbitMQ controle GitHub, sino que el proyecto use GitHub para el código y RabbitMQ para distribuir eventos relevantes: apertura de una PR, publicación de un juego, inicio de una sesión o finalización de una validación. Esto encaja muy bien con la lógica de arquitectura desacoplada que ya atraviesa toda la práctica.

Ejemplo de comando real para levantar RabbitMQ con panel de gestión:

```
docker run -d \  
  --name rabbitmq \  
  -p 5672:5672 \  
  -p 15672:15672 \  
  rabbitmq:3-management
```

Ejemplo de contexto real:

Cuando se abre una pull request, no hace falta que Laravel haga todo en ese momento. Puede publicar un evento y dejar que otro servicio procese la validación o la notificación.

Instalar un MCP para RabbitMQ

Aquí no hay un único servidor “oficial universal” del ecosistema RabbitMQ, pero sí hay implementaciones activas. Una de las más serias es `amazon-mq/mcp-server-rabbitmq`, que expone operaciones de RabbitMQ como herramientas MCP, envuelve las admin APIs del broker y puede usarse directamente desde PyPI con `uvx`.

También existe `kenliao94/mcp-server-rabbitmq`, pero el de Amazon MQ muestra una documentación más completa y una release más reciente en GitHub.

Ejemplo de configuración directa con `uvx`:

```
{  
  "mcpServers": {  
    "rabbitmq": {  
      "command": "uvx",  
      "args": [  
        "amq-mcp-server-rabbitmq@latest",  
        "--allow-mutative-tools"  
      ]  
    }  
  }  
}
```

Ejemplo de contexto real:

La IA puede listar colas, comprobar si existe una exchange para eventos del proyecto o verificar si una cola de validaciones está recibiendo mensajes.

Qué papel tendría RabbitMQ dentro de esta práctica

Dentro del proyecto, RabbitMQ no debería verse como “otra cosa más para instalar”, sino como una forma distinta de organizar el trabajo del sistema. Laravel sigue siendo el núcleo del CRM, de la autenticación y de la API; GitHub sigue siendo el centro del código y del control de versiones; RabbitMQ aparece cuando ciertos procesos conviene ejecutarlos de forma desacoplada o asíncrona. Esto ayuda a que el proyecto se parezca más a un entorno real y no a una aplicación monolítica donde todo ocurre dentro de la misma petición.

Ejemplo de publicación de un evento desde Laravel o desde un servicio auxiliar:

```
{  
  "event": "pull_request.opened",  
  "repository": "plataforma-juegos",  
  "branch": "feature/reverb-chat",  
  "author": "developer2"  
}
```

Ejemplo de contexto real:

Ese evento puede activar una revisión automática, una notificación al equipo o un registro interno de actividad sin bloquear el flujo principal.

Relación entre GitHub, MCP y RabbitMQ

Cuando se conectan los tres elementos, la arquitectura gana sentido pedagógico. GitHub mantiene el repositorio y el flujo de trabajo; MCP permite que un asistente entienda y opere sobre ese flujo; RabbitMQ permite que lo que ocurre en ese flujo genere eventos para otros procesos. Ya no hablamos solo de “subir versiones”, sino de una cadena de trabajo organizada, revisable y automatizable. Esto mejora de forma directa la parte de organización del proyecto que la práctica ya plantea en GitHub.

Ejemplo de arquitectura real:

GitHub -> GitHub MCP -> asistente IA -> Laravel / scripts internos -> RabbitMQ -> workers

Ejemplo de contexto real:

Una PR nueva entra en GitHub, el asistente la resume, comprueba si toca API o despliegue y envía un evento a RabbitMQ para lanzar pruebas o avisar al responsable del módulo.

La instalación de MCP para GitHub y de RabbitMQ como capa de eventos permite que el proyecto no se limite a almacenar código y ejecutar funcionalidades, sino que incorpore una forma de trabajo propia de entornos reales. GitHub centraliza el desarrollo, MCP añade una capa de apoyo inteligente sobre el repositorio y RabbitMQ permite distribuir eventos del sistema sin acoplar todos los procesos entre sí. De este modo, el alumnado no solo desarrolla una aplicación web, sino que aprende a mantenerla, revisarla y hacerla evolucionar con una organización más profesional.

Ejemplo de cierre realista:

*Repositorio en GitHub + revisión asistida con MCP + eventos en RabbitMQ
= proyecto más mantenible, más explicable y más cercano a producción.*

Nota sobre el código y las herramientas utilizadas

El código, comandos y configuraciones mostrados en esta sección tienen un carácter orientativo y se utilizan como ejemplo para facilitar la comprensión de los conceptos trabajados. No deben considerarse como una configuración definitiva ni única.

Se valorará <<<< especialmente >>>> la capacidad de contrastar la información, comprender qué se está haciendo y justificar las decisiones técnicas adoptadas, más allá de copiar directamente los ejemplos proporcionados.